# Python OOP and Intermediate Topics — Hands-on Practice in VS Code

---

# 1. Recommended folder structure in VS Code

Create one folder:

Python_OOP_Intermediate_Lab

Inside it, create these files:

01_classes_objects.py
02_instance_variables_constructor.py
03_inheritance.py
04_polymorphism_encapsulation.py
05_class_static_special_methods.py
06_iterators_generators.py
07_decorators.py
08_higher_order_map_filter_reduce.py
09_json_handling.py
10_basic_regex.py
11_mini_project.py

Open this folder in VS Code.

Run each file one by one.

---

# 2. Lab 1 — Classes and Objects

File: `01_classes_objects.py`

```
class Car:
    pass
```

```
car1 = Car()
car2 = Car()

print("car1:", car1)
print("car2:", car2)
```

## Explanation

Here:

- `Car` is the class
- `car1` and `car2` are objects

Even though the class is empty, Python still allows object creation.

## Run in VS Code terminal

python 01_classes_objects.py

## What you learn

- how to define a class
- how to create objects

---

## Improved example

```
class Car:
  def show_message(self):
      print("This is a car object")


car1 = Car()
car2 = Car()

car1.show_message()
car2.show_message()
```

## Explanation

`show_message()` is an instance method.
It is called using the object.

---

## Practice task

Create a class `Mobile` with a method `show_brand()` that prints:

This is a mobile

---

# 3. Lab 2 — Instance Variables and Constructor

File: `02_instance_variables_constructor.py`

```
class Student:
  def __init__(self, name, age, course):
      self.name = name
      self.age = age
      self.course = course

  def display(self):
      print("Name   :", self.name)
      print("Age    :", self.age)
      print("Course :", self.course)


s1 = Student("Giri", 25, "Python")
s2 = Student("Anu", 22, ".NET")

s1.display()
print("-" * 30)
s2.display()
```

## Explanation

- `__init__()` is the constructor
- `self.name`, `self.age`, `self.course` are instance variables

- each object stores its own values

## Expected output

```
Name   : Giri
Age    : 25
Course : Python
------------------------------
Name   : Anu
Age    : 22
Course : .NET
```

---

## Why `self` is needed

`self` means the current object.

For `s1`:

self.name = "Giri"

For `s2`:

self.name = "Anu"

That is why different objects keep separate values.

---

## Practice task

Create an `Employee` class with:

- `emp_id`
- `name`
- `salary`

Add a method `show_details()`.

---

# 4. Lab 3 — Destructor

You asked about destructors also, so let us see a simple example.

Add this in a file or test separately:

```
class Demo:
  def __init__(self, name):
      self.name = name
      print(f"{self.name} object created")

  def __del__(self):
      print(f"{self.name} object destroyed")


d1 = Demo("Test")
del d1
```

## Explanation

- `__init__()` runs when object is created
- `__del__()` may run when object is deleted

## Important note

In Python, destructor timing is not always guaranteed exactly when you expect, because of garbage collection.

So in real Python coding:

- destructor is not used very often
- context managers and explicit cleanup are preferred

---

# 5. Lab 4 — Inheritance

File: `03_inheritance.py`

```
class Person:
  def __init__(self, name):
```

```
        self.name = name

    def show_name(self):
        print("Name:", self.name)


class Teacher(Person):
    def teach(self):
        print(self.name, "is teaching Python")


class Student(Person):
    def study(self):
        print(self.name, "is studying Python")


t1 = Teacher("Ravi")
s1 = Student("Meena")

t1.show_name()
t1.teach()

print("-" * 30)

s1.show_name()
s1.study()
```

## Explanation

- `Person` is the parent class
- `Teacher` and `Student` are child classes
- both child classes inherit `show_name()`

## Expected output

```
Name: Ravi
Ravi is teaching Python
------------------------------
Name: Meena
Meena is studying Python
```

---

## Using `super()`

```
class Person:
  def __init__(self, name):
      self.name = name


class Employee(Person):
  def __init__(self, name, salary):
      super().__init__(name)
      self.salary = salary

  def display(self):
      print("Name:", self.name)
      print("Salary:", self.salary)


e1 = Employee("Kumar", 50000)
e1.display()
```

## Why `super()`?

It helps call the parent class constructor or methods.

---

## Practice task

Create:

- parent class `Vehicle`
- child class `Bike`
- child class `Car`

Add separate methods in child classes.

---

# 6. Lab 5 — Polymorphism

File: `04_polymorphism_encapsulation.py`

First part: polymorphism

```
class Animal:
  def sound(self):
      print("Animal makes a sound")


class Dog(Animal):
  def sound(self):
      print("Dog barks")


class Cat(Animal):
  def sound(self):
      print("Cat meows")


animals = [Animal(), Dog(), Cat()]

for a in animals:
  a.sound()
```

## Explanation

All classes have a `sound()` method.
But the behavior changes depending on object type.

This is **method overriding**, a common form of polymorphism.

## Expected output

Animal makes a sound
Dog barks
Cat meows

---

## Another polymorphism example

```
class ExcelReport:
  def export(self):
      print("Exporting report as Excel")


class PdfReport:
```

```
    def export(self):
        print("Exporting report as PDF")


def generate_report(report):
    report.export()


generate_report(ExcelReport())
generate_report(PdfReport())
```

## Explanation

The same function `generate_report()` works with different object types.
That is a flexible OOP design.

---

# 7. Lab 6 — Encapsulation

Continue in `04_polymorphism_encapsulation.py`

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Deposited:", amount)
        else:
            print("Invalid deposit")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal")
        elif amount > self.__balance:
            print("Insufficient balance")
        else:
            self.__balance -= amount
            print("Withdrawn:", amount)
```

```
    def get_balance(self):
        return self.__balance


acc = BankAccount("Giri", 10000)

acc.deposit(2000)
acc.withdraw(3000)
print("Available Balance:", acc.get_balance())
```

## Explanation

`__balance` is private.
It should not be directly changed from outside.

Instead, the class controls it through:

- `deposit()`
- `withdraw()`
- `get_balance()`

## Output

Deposited: 2000
Withdrawn: 3000
Available Balance: 9000

---

## Wrong approach

acc.__balance = -50000

This should not be done.

Encapsulation prevents uncontrolled modification.

---

## Practice task

Create a class `StudentMarks` with:

- private variable `__marks`
- method `set_marks()`
- method `get_marks()`

Do not allow marks below 0 or above 100.

---

# 8. Lab 7 — Class Methods, Static Methods, Special Methods

File: `05_class_static_special_methods.py`

---

## 8.1 Class variable and class method

```
class Employee:
  company_name = "ABC Technologies"

  def __init__(self, name):
      self.name = name

  def show(self):
      print(f"Employee Name: {self.name}, Company: {Employee.company_name}")

  @classmethod
  def change_company(cls, new_name):
      cls.company_name = new_name


e1 = Employee("Giri")
e2 = Employee("Anu")

e1.show()
e2.show()

Employee.change_company("XYZ Solutions")

e1.show()
e2.show()
```

## Explanation

- `company_name` belongs to the class, not individual objects
- `@classmethod` works with class-level data

---

## 8.2 Static method

```
class Calculator:
  @staticmethod
  def add(a, b):
      return a + b

  @staticmethod
  def multiply(a, b):
      return a * b


print(Calculator.add(10, 20))
print(Calculator.multiply(5, 4))
```

## Explanation

Static methods are utility methods.
They do not use `self` or `cls`.

---

## 8.3 Special method `__str__`

```
class Student:
  def __init__(self, name, course):
      self.name = name
      self.course = course

  def __str__(self):
      return f"Student(name={self.name}, course={self.course})"


s1 = Student("Giri", "Python")
print(s1)
```

## Output

Student(name=Giri, course=Python)

Without `__str__`, Python prints an internal object address.

---

## 8.4 Special method `__len__`

```
class Team:
  def __init__(self, members):
      self.members = members

  def __len__(self):
      return len(self.members)


team1 = Team(["A", "B", "C", "D"])
print(len(team1))
```

## Output

4

---

## 8.5 Special method `__add__`

```
class Number:
  def __init__(self, value):
      self.value = value

  def __add__(self, other):
      return self.value + other.value


n1 = Number(10)
n2 = Number(15)

print(n1 + n2)
```

## Output

25

This is operator overloading.

---

# 9. Lab 8 — Iterators

File: `06_iterators_generators.py`

```
numbers = [100, 200, 300]

it = iter(numbers)

print(next(it))
print(next(it))
print(next(it))
```

## Explanation

- `iter(numbers)` creates an iterator
- `next(it)` gets values one by one

---

## Custom iterator example

```
class Counter:
  def __init__(self, limit):
      self.limit = limit
      self.current = 1

  def __iter__(self):
      return self

  def __next__(self):
      if self.current <= self.limit:
          value = self.current
          self.current += 1
          return value
      raise StopIteration


counter = Counter(5)
```

```
for num in counter:
  print(num)
```

## Output

```
1
2
3
4
5
```

## Explanation

To create your own iterator:

- define `__iter__()`
- define `__next__()`

---

# 10. Lab 9 — Generators

Continue in `06_iterators_generators.py`

```
def number_generator(limit):
  for i in range(1, limit + 1):
      yield i


for value in number_generator(5):
  print(value)
```

## Explanation

A generator is a simpler way to create an iterator.
Instead of storing everything at once, it yields one value at a time.

---

## Generator example with squares

```
def square_generator(limit):
    for i in range(1, limit + 1):
        yield i * i


for sq in square_generator(5):
    print(sq)
```

## Output

```
1
4
9
16
25
```

---

## Why generators are powerful

Suppose you want 10 lakh values.
A normal list stores all values in memory.
A generator produces values only when needed.

That makes generators memory efficient.

---

# 11. Lab 10 — Decorators

File: `07_decorators.py`

```
def simple_decorator(func):
    def wrapper():
        print("Before function execution")
        func()
        print("After function execution")
    return wrapper


@simple_decorator
def say_hello():
    print("Hello from function")
```

```
say_hello()
```

## Output

```
Before function execution
Hello from function
After function execution
```

---

## Decorator with arguments

```
def log_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Function {func.__name__} completed")
        return result
    return wrapper


@log_decorator
def add(a, b):
    return a + b


print(add(10, 5))
```

## Explanation

When your function accepts parameters, decorator wrapper should usually use:

- `*args`
- `**kwargs`

---

## Real-world use

Decorators are commonly used for:

- logging

- authorization
- timing
- caching
- validation

---

## Practice task

Write a decorator that prints:

- `"Start"`
- executes the function
- `"End"`

---

# 12. Lab 11 — Higher-Order Functions

File: `08_higher_order_map_filter_reduce.py`

```
def greet(name):
  return f"Hello, {name}"


def process_user(func, name):
  return func(name)


print(process_user(greet, "Giri"))
```

## Explanation

A higher-order function:

- takes another function as input
- or returns another function

---

## Returning a function

```
def outer():
  def inner():
      print("Inside inner function")
  return inner


result = outer()
result()
```

---

## Why important?

Because:

- decorators use this concept
- functional programming uses this concept
- Python callbacks and dynamic behavior use this concept

---

# 13. Lab 12 — map(), filter(), reduce()

Continue in `08_higher_order_map_filter_reduce.py`

---

## 13.1 map()

```
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, numbers))
print("Squares:", squares)
```

## Output

Squares: [1, 4, 9, 16, 25]

---

## 13.2 filter()

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print("Even Numbers:", even_numbers)
```

# Output

Even Numbers: [2, 4, 6]

---

# 13.3 reduce()

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

total = reduce(lambda a, b: a + b, numbers)
print("Sum:", total)
```

# Output

Sum: 15

---

# Real-world examples

## map

Convert names to uppercase

```
names = ["giri", "anu", "ravi"]
upper_names = list(map(str.upper, names))
print(upper_names)
```

## filter

Select salaries greater than 50000

```
salaries = [25000, 50000, 75000, 90000]
high_salaries = list(filter(lambda s: s > 50000, salaries))
print(high_salaries)
```

## reduce

Find total salary

```
from functools import reduce

salaries = [25000, 50000, 75000]
total_salary = reduce(lambda a, b: a + b, salaries)
print(total_salary)
```

---

## Important note

In Python, many developers prefer:

- loops
- list comprehensions

because they are often easier to read than complex `map/filter/reduce`.

Example:

```
numbers = [1, 2, 3, 4]
squares = [x * x for x in numbers]
print(squares)
```

---

# 14. Lab 13 — JSON Handling

File: `09_json_handling.py`

---

## 14.1 Convert Python dictionary to JSON string

```
import json

student = {
  "id": 101,
  "name": "Giri",
  "course": "Python",
  "fees_paid": True
}

json_data = json.dumps(student, indent=4)
print(json_data)
```

## Explanation

- `json.dumps()` converts Python object to JSON string
- `indent=4` makes it readable

---

## 14.2 Convert JSON string to Python dictionary

```
import json

json_text = '{"name": "Anu", "age": 24, "city": "Chennai"}'

data = json.loads(json_text)

print(data)
print(data["name"])
print(type(data))
```

## Output

```
{'name': 'Anu', 'age': 24, 'city': 'Chennai'}
Anu
<class 'dict'>
```

---

## 14.3 Write JSON to file

```
import json

product = {
  "product_id": 1,
  "product_name": "Laptop",
  "price": 55000
}

with open("product.json", "w") as file:
  json.dump(product, file, indent=4)

print("JSON file created successfully")
```

---

## 14.4 Read JSON from file

```
import json

with open("product.json", "r") as file:
  data = json.load(file)

print(data)
print(data["product_name"])
```

---

## Where JSON is used in real projects

- REST API request/response
- configuration files
- frontend/backend communication
- data storage and integration

---

# 15. Lab 14 — Basic Regex

File: `10_basic_regex.py`

---

## 15.1 Search first number in string

```
import re

text = "Order ID is 45231"
match = re.search(r"\d+", text)

if match:
  print("Found number:", match.group())
```

---

## 15.2 Find all numbers

```
import re

text = "Marks are 78, 88 and 95"
numbers = re.findall(r"\d+", text)
```

```
print(numbers)
```

## Output

['78', '88', '95']

---

## 15.3 Validate email

```
import re

email = "demo123@gmail.com"
pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"

if re.match(pattern, email):
  print("Valid email")
else:
  print("Invalid email")
```

---

## 15.4 Replace text

```
import re

text = "Python is easy and Python is powerful"
new_text = re.sub(r"Python", "Java", text)

print(new_text)
```

---

## 15.5 Extract phone numbers

```
import re

text = "Contact numbers: 9876543210, 9123456780"
phones = re.findall(r"\d{10}", text)

print(phones)
```

---

## Common regex symbols

**`\d`**

digit

`\w`

word character

`\s`

space

`+`

one or more

`*`

zero or more

`^`

start of string

`$`

end of string

`{10}`

exactly 10 times

---

## Practice task

Write regex to:

- extract all emails from text
- extract all 10-digit mobile numbers
- validate PIN code of 6 digits

---

# 16. Mini Project — Student Record Processor

File: `11_mini_project.py`

This small project combines:

- class
- constructor
- encapsulation
- special method
- JSON
- regex
- map/filter

```
import json
import re


class Student:
    school_name = "ABC Academy"

    def __init__(self, student_id, name, email, marks):
        self.student_id = student_id
        self.name = name
        self.email = email
        self.__marks = marks

    def is_valid_email(self):
        pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        return re.match(pattern, self.email) is not None

    def get_marks(self):
        return self.__marks

    def set_marks(self, new_marks):
        if 0 <= new_marks <= 100:
            self.__marks = new_marks
        else:
            print("Invalid marks")

    def is_pass(self):
        return self.__marks >= 40
```

```
    def to_dict(self):
        return {
            "student_id": self.student_id,
            "name": self.name,
            "email": self.email,
            "marks": self.__marks
        }

    def __str__(self):
                        return    f"Student(id={self.student_id},    name={self.name},    email={self.email},
marks={self.__marks})"


students = [
    Student(1, "Giri", "giri@gmail.com", 75),
    Student(2, "Anu", "anu@gmail.com", 35),
    Student(3, "Ravi", "ravi123@gmail.com", 82)
]

print("----- ALL STUDENTS -----")
for student in students:
    print(student)

print("\n----- VALID EMAIL CHECK -----")
for student in students:
    print(student.name, ":", student.is_valid_email())

print("\n----- PASS STUDENTS -----")
passed_students = list(filter(lambda s: s.is_pass(), students))
for student in passed_students:
    print(student.name, "-", student.get_marks())

print("\n----- STUDENT NAMES IN UPPERCASE -----")
upper_names = list(map(lambda s: s.name.upper(), students))
print(upper_names)

print("\n----- SAVE TO JSON FILE -----")
student_data = [student.to_dict() for student in students]

with open("students.json", "w") as file:
    json.dump(student_data, file, indent=4)

print("students.json file created successfully")
```

---

## What this project teaches

- how to create a class
- how to use constructor
- how to protect data with encapsulation
- how to validate email with regex
- how to use `filter()` to get pass students
- how to use `map()` to transform names
- how to store output in JSON file

---

# 17. Best order to practice in VS Code

Practice in this sequence:

## Stage 1

- classes
- objects
- instance variables
- constructor

## Stage 2

- inheritance
- polymorphism
- encapsulation

## Stage 3

- class methods
- static methods
- special methods

## Stage 4

- iterators
- generators

## Stage 5

- decorators
- higher-order functions
- map/filter/reduce

## Stage 6

- JSON handling
- basic regex

That order helps because each topic builds on the previous one.

---

# 18. Common interview-style questions

## Classes and Objects

**Question:** What is the difference between class and object?
**Answer:** Class is a blueprint; object is an instance of that blueprint.

## Constructor

**Question:** What is `__init__()` in Python?
**Answer:** It is a special method automatically called when an object is created.

## Encapsulation

**Question:** How do you make a variable private in Python?
**Answer:** By prefixing it with double underscore, like `__balance`.

## Inheritance

**Question:** Why use inheritance?
**Answer:** To reuse code from an existing class and extend functionality.

## Polymorphism

**Question:** What is polymorphism in Python?
**Answer:** Same method name behaving differently for different object types.

## Decorators

**Question:** What is a decorator?
**Answer:** A function that wraps another function and adds extra behavior.

## Generator

**Question:** Why use generators?
**Answer:** They save memory by producing one value at a time.

## JSON

**Question:** Difference between `json.dump()` and `json.dumps()`?
**Answer:** `dump()` writes to file, `dumps()` returns a JSON string.

## Regex

**Question:** What does `\d+` mean?
**Answer:** One or more digits.

---

# 19. Real-world relevance

These are not just theory topics.

They are used in real work like:

- **OOP** → application design, models, services
- **Encapsulation** → protect business logic
- **Inheritance/Polymorphism** → extensible systems
- **Decorators** → logging, security, timing
- **Generators** → large data processing
- **JSON** → APIs, web apps, config files
- **Regex** → validation, parsing, extraction

For example, in API development:

- response comes in JSON
- email/mobile validation uses regex
- entities are represented as classes
- decorators can be used for logging or access control

---

# 20. Strong recommendation for you

Do not only read these programs.
Type them yourself in VS Code.

That is the fastest way to understand:

- indentation
- syntax
- object creation
- method calls
- imports
- file execution

When you type and run, you will also see common errors like:

- indentation error
- missing colon
- wrong `self`
- wrong decorator wrapper
- missing import for `reduce`
- wrong JSON syntax

Those mistakes are part of learning.

---

# 21. Practice exercises for you

Try these on your own in VS Code:

### Exercise 1

Create a `Book` class with:

- title
- author
- price
- method to display details

## Exercise 2

Create parent class `Shape` and child classes `Circle` and `Rectangle`.

## Exercise 3

Create a `BankAccount` class with private balance and deposit/withdraw methods.

## Exercise 4

Create a generator that yields even numbers from 1 to 20.

## Exercise 5

Create a decorator that prints execution start and end.

## Exercise 6

Use `map()` to convert list of names to lowercase.

## Exercise 7

Use `filter()` to get students scoring above 60.

## Exercise 8

Store employee details in JSON file and read them back.

## Exercise 9

Use regex to validate:

- email
- mobile number
- pincode

---

# 22. Expert insight

For Python learners, the most important shift is this:

At beginner level, you write code that works.
At intermediate level, you write code that is:

- reusable
- organized
- readable
- extendable

That is exactly why these topics matter.

A good Python learner should not treat these as separate chapters only.
You should gradually combine them.

For example:

- create classes
- read data from JSON
- validate using regex
- transform using `map/filter`
- lazily process records using generator
- add logging using decorator

That is closer to real development.